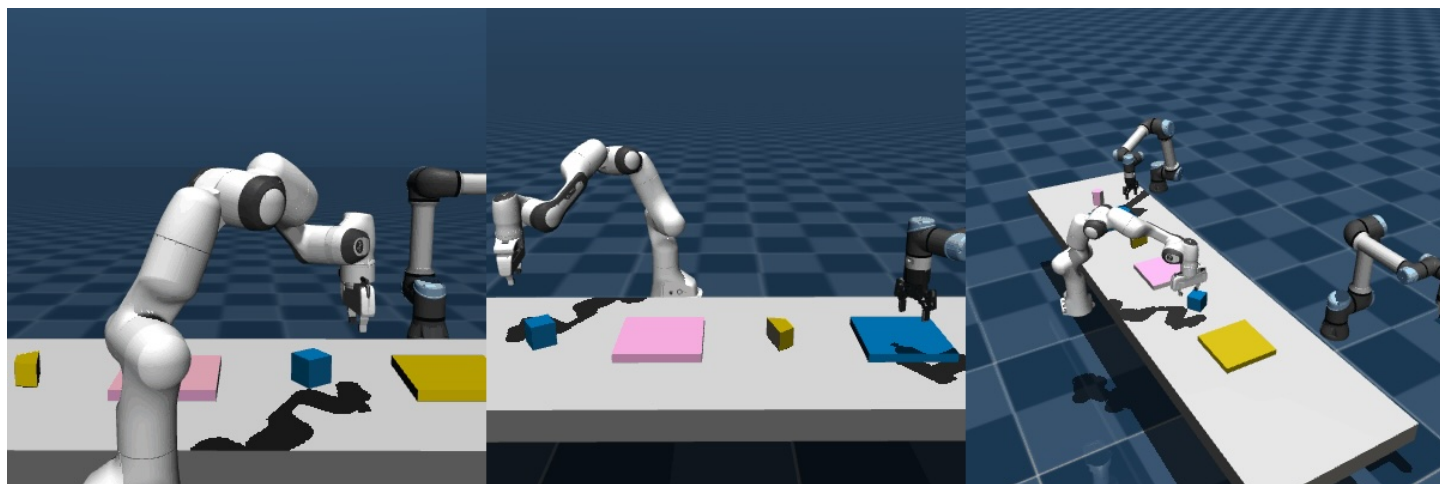


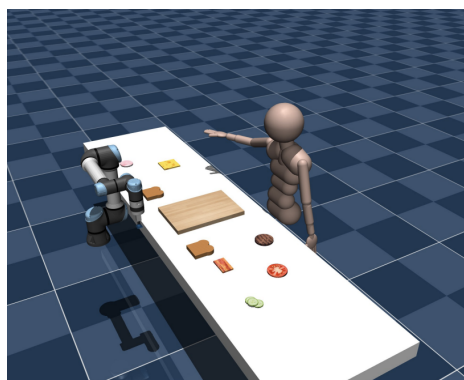
题目：多机器人协同的具身操作

任务介绍

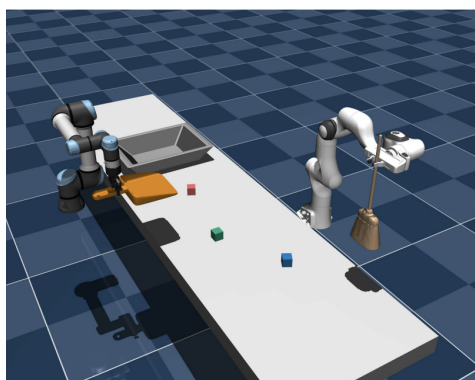
任务简介：在基于Mujoco的多机器人操作任务的仿真环境(RocoBench)中，基于大模型实现多机器人协同合作，相互交流完成指定的机器人操作任务。



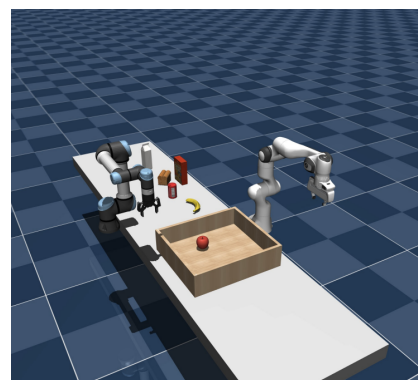
如下是任务环境集合里的一些任务：



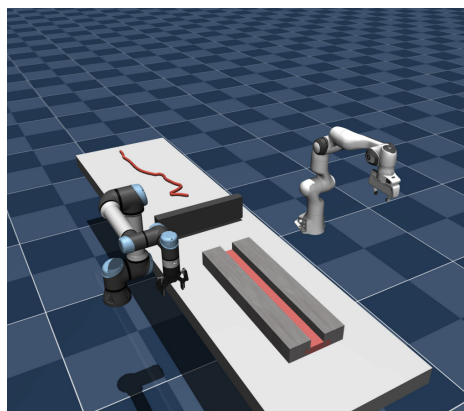
Make Sandwich



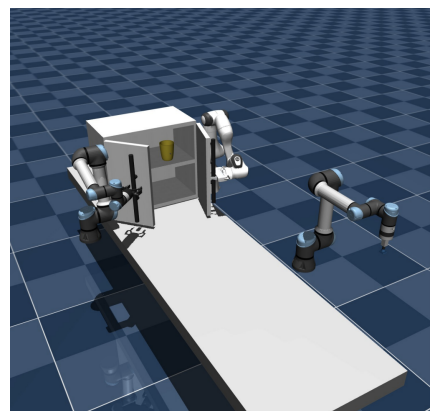
Sweep Floor



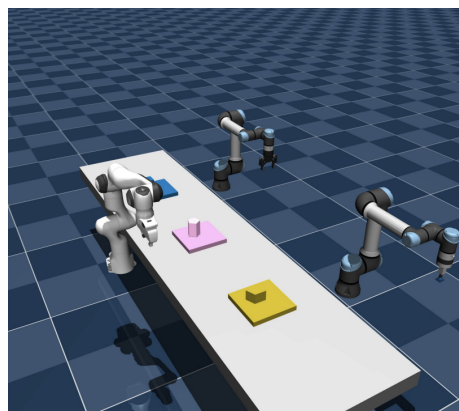
Pack Grocery



Move Rope



Arrange Cabinet



Sort Cube

1. Sweep Floor

两个机器人利用簸箕和一把扫帚进行清扫，然后拿着垃圾桶的机器人将立方体倾倒到垃圾桶中。

2. Make Sandwich

两个机器人一起做三明治，每个机器人都可以使用不同的配料。他们必须选择所需的物品并轮流以正确的顺序堆叠它们。

3. Sort Cubes

机器人将3个立方体分类到相应的面板上。机器人必须保持在各自的范围内，并互相帮助将立方体移动到合理的位置

4. Pack Grocery

两个机器人将桌子上的一组杂货打包到垃圾箱中。机器人需要协调它们的路径以避免碰撞。

5. Move Rope

两个机器人将绳子一起举起越过墙壁并将其放入凹槽中。他们必须协调他们的抓手以避免碰撞。

6. Arrange Cabinet

3个机器人，其中两个各自保持柜门的一侧打开，而第三个机器人将杯子取出并将它们放在正确的杯垫上

任务使用机器人： UR5E机械臂 / Franka Panda机械臂 / Humanoid 等；

任务观测： 根据具体任务而定，比如清扫任务中，包含方块位置（桌子上/簸箕里/垃圾桶里），机器人状态（夹爪3D位置）；

任务技能空间： 根据具体任务而定，比如清扫任务中，包含MOVE [target]：目标物体移动；SWEEP [target]：目标物体清扫到簸箕中；WAIT 等待；DUMP：物体倒到垃圾桶中。

有关任务的更多细节，可以参考rocobench/envs/task_TASKNAME.py文件中，任务信息在与大模型对话时作为prompt输入给大模型。

环境配置

选择一：使用官方配置好的镜像

在创建实例的时候，选择公开可见镜像：`ubuntu2204-roco-exam:1.0`，注意名称需**完全一致**，进入实例，激活已经创建好的名为 `roco` 的conda环境。

选择二：手动配置

需在可上网区配置环境，建议使用Ubuntu 22.04系统和Python 3.8环境。在下载压缩包并解压后，在项目文件夹内启动终端：

创建虚拟环境：

代码块

```
1 conda create -n robot python=3.8
```

安装依赖

代码块

```
1 conda activate robot
2 pip install -r requirements.txt
```

测试运行任务:

代码块

```
1 python run_dialog.py
```

运行到大模型对话部分出错是正常的，因为代码没有实现大模型对话的接口；如果出现import报错说明环境配置有问题。

任务要求

接入大模型，完成机器人任务

1. 部署大模型

请大家自行部署大模型，建议使用Ollama等工具进行部署（<https://github.com/ollama/ollama/>）

 **注意：**

- 本次任务限定使用大小在80b以下的大模型，比如 llama3.3 70b大模型；
- 模型下载等操作可能需要开启可上网区实例进行操作；
- 在可上网区下载ollama模型时注意，建议修改 OLLAMA_MODELS 变量（`export OLLAMA_MODELS="YOURPATH"`），将模型保存到账号的存储区，在GPU区可以用ollama直接加载就使用。如果保存到默认位置，系统打包镜像时会被打包进去。

2. 在代码框架中对接部署好的大模型

代码框架支持多种多机器人合作的交流模式，对应 `run_dialog.py` 中的 `comm_mode` 参数的不同设置。

- 若进行多机器人对话式规划，比如 `comm_mode=dialog`，需要在 `dialog_prompter.py` 第 **264-267**行处，对接大模型接口；
- 若进行集中式规划，比如 `comm_mode=plan`，需要在 `plan_prompter.py` 第**245-248**行处，对接大模型接口；获取并解析response完成指定机器人操作任务；

本次项目不限制使用某种模式，也可以修改机器人交互合作的流程，自定义新的方法。

注意：

- Ollama-python依赖pydantic>2，可能会与当前代码框架发生冲突。可以使用Ollama的 [openai接口（OpenAI compatibility - Ollama）](#) 进行通信。
- python版本及openai库版本较低，注意接口使用。

3. 运行run_dialog.py完成指定任务

代码块

```
1 python run_dialog.py --task TASK_NAME --num_runs NUM_RUNS --skip_display
```

其中TASK_NAME为测试的任务名称，NUM_RUNS为任务测试的次数。更多参数详见 `run_dialog.py`，可选的TASK_NAME可以在 `run_dialog.py` 里看到具体定义。

- 运行代码时使用conda的roco环境，可以使用xvfb 创建虚拟显示器：

代码块

```
1 xvfb-run python xxx.py
```

提高任务的成功率

目前测试下来可能在一些任务上效果并不太好，体现在成功率上会比较低。请大家分析当前的结果，为什么会出现机器人操作效果不好的情况，并进行改进。

为了大家测试的方便，项目提供了 `evaluator.py` 文件，会**自动测试多个任务上**的成功率等结果。

代码块

```
1 python evaluator.py
```

可能的改进效果的思路：

1. 设计更加优质的prompt，让机器人更加明确任务

2. 改进多机器人协同机制，机器人之间如何更好地交流合作，遇到错误进行反思等
3. 改进路径规划机制，让机器人在运动过程中减少碰撞等带来的成功率损失
4. 对大模型进行finetune等

任务时间限制

每个任务的每个run的运行时间限制在10 min之内，超时则认为失败。

提交材料

1. 项目报告（**小组**）：说明项目的具体工作，包括完成的流程，改进效果的方法，项目结果以及对应的截图材料。
2. 项目报告（**个人**）：**明确每个人的分工，做了什么贡献。**
3. 代码：项目的代码材料（**不要提交log等大文件**），包括环境的配置信息，要求可以复现项目的结果。
4. 实例jupyter server link：实例需保持运行状态，保留可运行环境，助教需进入实例运行 `evaluator.py`，测评成功率结果。

